External Validation of a Positive-Closure Proof Architecture

A Multi-Tranche Survey: Quotients, Products, and Dependent Sums in Lean

Nova Spivack

2026

Abstract

The Infinity Compression (IC) formalization develops a layered positive-closure proof architecture: collapse to a bare certificate, a typed forgetful map, fiber structure over that certificate, and, where appropriate, canonical witnesses or sections. A natural methodological concern is that this architecture may be endogenous to the flagship NEMS-style case rather than a reusable proof pattern. This paper presents a **consolidated machine-checked external validation**: three independent tranches of ordinary mathematics, each formalized in Lean without importing the IC spine, each comparing a *flat baseline* (the obvious Mathlib-shaped existence or representability statement) with an *architecture-guided* theorem bundle (nonempty fibers, surjectivity of the forgetful map, `HasRightInverse`, an identified `RightInverse` section, and a uniform canonical witness in each fiber).

Tranche **T1** is the setoid quotient benchmark (collapse by an equivalence relation; baseline `Quotient.exists_rep`). Tranche **T2** is the Cartesian product with first projection (many enriched pairs over one bare image—*multi-route* without quotienting). Tranche **T3** is the dependent sum $\Sigma_{b:B} E\, b$ with `Sigma.fst` (bundle projection; dependent generalization of T2). The contribution is not a catalog of new classical theorems; it is a demonstration that the same architectural lens organizes *all three* external families and yields strengthened, reusable proof interfaces in each case. Under the project's validation rubric, the combined evidence supports a unified **STRONG** verdict for methodological transferability. We situate the work in the NEMS / APS / IC / Strata stack and state explicitly that APS is not tested by these tranches.

Contents

1 Introduction **3**

2 The positive-closure architecture as a reusable schema **3**

3 Unified methodology: baseline vs. architecture-guided bundle **3**

4 Tranche T1: Setoid quotients **4**
- 4.1 Why quotients . . . 4
- 4.2 Lean formalization and main bundle . . . 4
- 4.3 What the architectural packaging adds . . . 4

5 Tranche T2: Cartesian product and first projection **4**
- 5.1 Mathematical story . . . 4
- 5.2 Baseline vs. bundle . . . 4

6 Tranche T3: Dependent sums and the bundle projection **5**
6.1 Mathematical story 5
6.2 Hypothesis and classical section 5

7 Comparative synthesis across tranches **5**

8 Unified validation verdict and the meaning of STRONG **6**

9 Place in the larger program **6**

10 Limitations and future work **6**

A Theorem indices (Lean names by module) **6**

1 Introduction

The Infinity Compression (IC) program develops a positive-closure proof architecture organized around four layers: a collapsed bare layer, a projection from enriched to bare structure, fibers over bare points, and, when available, canonical witnesses or sections. In the flagship IC formalization this architecture is load-bearing: it organizes the distinction between theorem-level certification and richer reflective realization. **The central claim of this paper is that the positive-closure architecture is a reusable proof pattern—not merely a custom narrative for the flagship case—and that claim is supported by multiple independent external benchmarks.**

A single quotient formalization already begins to answer the worry that the architecture might be idiosyncratic; this document *consolidates* that quotient tranche with two additional standard constructions—Cartesian products and dependent sums—so that the evidence base is explicitly multi-family. The result is one self-contained validation paper: readers need not cross-reference a separate "survey" manuscript for the product and bundle stories.

Scope. We do not claim a universal theorem that every closure problem in mathematics must be formalized in IC's types. We claim that, in Lean, on three concrete external families, the architecture exports cleanly, yields structured theorem bundles beyond flat baselines, and aligns with the flagship forgetful / fiber / section narrative [1].

2 The positive-closure architecture as a reusable schema

Let π denote a forgetful projection from an *enriched* carrier to a *bare* certificate space (a collapsed layer where theorem-level records live). Write E for an enriched carrier and B for a bare layer. The schema is:

$$E \xrightarrow{\pi} B, \qquad \mathrm{Fib}(b) \coloneqq \pi^{-1}(b) \ \text{ for } b \in B, \qquad \text{optionally } s : B \to E \text{ with } \pi \circ s = \mathrm{id}_B. \tag{1}$$

Here $\mathrm{Fib}(b)$ is the fiber over a bare point; a section s is a global choice of preimage.

Each tranche below instantiates (1) with different E, B, and π. The quotient tranche uses Mathlib's quotient map and `Quotient.out`; the product tranche uses first projection and a parametric section $x \mapsto (x, y_0)$; the bundle tranche uses `Sigma.fst` and a `noncomputable` section from classical choice when $\forall b,\ \mathrm{Nonempty}(E\,b)$.

3 Unified methodology: baseline vs. architecture-guided bundle

In every tranche we record:

- a **baseline** statement profile matching how Mathlib users typically see the mathematics (existence of representatives, trivial points in fibers, etc.);
- an **architecture-guided bundle**: named bare layer, named projection, fiber type as a subtype or structured copy, surjectivity of π, `HasRightInverse`, a concrete `RightInverse` witness, and a `canonical...Witness` in each fiber.

The methodological gain is *interface structure*: the bundle is a reusable template parallel to the flagship story, not a claim of new classical content.

4 Tranche T1: Setoid quotients

4.1 Why quotients

This tranche is a positive-closure benchmark in the sense relevant to IC: it exhibits collapse, fibers, and a uniform section-like recovery mechanism (`Quotient.out`), not only an existential shadow.

The quotient construction is standard mathematics (see, e.g., [6]); it has a genuine collapse map from carriers to quotient points; it admits a classical baseline (`Quotient.exists_rep`); and it supports a section story via `Quotient.out`. The benchmark imports only `Mathlib.Data.Setoid.Basic` and `Mathlib.Logic.Function.Basic`.

4.2 Lean formalization and main bundle

All definitions and theorems live in `InfinityCompression/Validation/QuotientFiberBenchmark.lean` (namespace `InfinityCompression.Validation`). Layering: `BareQuotient`, `EnrichedCarrier`, `forgetToQuotient`, `QuotientFiber`, `canonicalQuotientFiberWitness`.

Lean name	Meaning	Baseline vs. architecture
`quotient_fiber_nonempty`	each quotient point has a witness in its `QuotientFiber`	baseline-plus (typed fiber nonempty)
`forgetToQuotient_surjective`	`forgetToQuotient` hits every bare point	architecture
`forgetToQuotient_hasRightInverse`	projection admits a global right inverse	architecture
`quotientOut_rightInverse_forgetToQuotient`	`Quotient.out` is a `RightInverse` of `forgetToQuotient`	architecture
`canonicalQuotientFiberWitness`	uniform canonical witness in each fiber	architecture

Table 1: T1 (quotient): main named results in the architecture-guided bundle.

4.3 What the architectural packaging adds

Classically, every quotient class has *some* representative (`Quotient.exists_rep`). The architecture-guided view reorganizes the same fact under (1): surjectivity, `HasRightInverse`, identification of `Quotient.out` as `RightInverse`, and `canonicalQuotientFiberWitness`. Methodologically, this upgrades a pointwise existential into a named, reusable interface—the same move the positive-closure architecture is designed to expose internally.

5 Tranche T2: Cartesian product and first projection

5.1 Mathematical story

Let X and Y be types. The first projection $\pi : X \times Y \to X$ forgets the second coordinate. For fixed nonempty Y, many pairs (x, y_1) and (x, y_2) share the same bare image x: this is *multi-route over one bare certificate* without any quotient—a clean stress test for "fiber + section" language distinct from equivalence collapsing.

5.2 Baseline vs. bundle

Baseline: once Y is inhabited, each $x : X$ has some pair mapping to it (e.g. (x, y_0) for a chosen y_0). **Architecture bundle:** we package `ProductFiberAt X Y x`, surjectivity of `forgetFirst`, `HasRightInverse`, the parametric section `productSection y0`, `RightInverse` compatibility, and

canonicalProductFiberWitness. The Lean module is ProductProjectionFiberBenchmark.lean; it imports only Mathlib.Logic.Function.Basic.

Lean name	One-line reading
forgetFirst	first projection $X \times Y \to X$.
ProductFiberAtXYx	fiber of forgetFirst over x (Y explicit in the name).
product_fiber_nonempty	each bare x has a nonempty fiber (given y0:Y).
forgetFirst_surjective	forgetFirst is surjective (given y0).
forgetFirst_hasRightInverse	surjectivity ⇒ HasRightInverse.
productSection_rightInverse_forgetFirst	productSectiony0 is RightInverse to forgetFirst.
canonicalProductFiberWitness	distinguished point â§Í(x,y0),rflâ§Ï in each fiber.

Table 2: T2 (product projection): architecture-guided bundle.

6 Tranche T3: Dependent sums and the bundle projection

6.1 Mathematical story

For $E : B \to \text{Type}$, the dependent sum $\Sigma_{b:B} E\, b$ is the standard *bundle* total space; Sigma.fst projects to the base. The fiber over $b : B$ is (equivalent to) $E\, b$. This generalizes T2: product is the special case of a constant family $E\, b = Y$.

6.2 Hypothesis and classical section

We assume $\forall b : B,\ \text{Nonempty}(E\, b)$ so that Sigma.fst is surjective. The global section sigmaSection uses Classical.choose per fiber; the module is noncomputable where required. Baseline: each base point has *some* fiber point. Bundle: SigmaFiber, surjectivity, HasRightInverse, sigmaSection_rightInverse_sigma_fst, canonicalSigmaFiberWitness.

Lean name	One-line reading
SigmaFiberEb	points in $\Sigma\, E$ lying over b.
sigma_fiber_nonempty	nonempty fibers from $\forall b,\ \text{Nonempty}(E\, b)$.
sigma_fst_surjective	Sigma.fst surjective.
sigma_fst_hasRightInverse	HasRightInverse for Sigma.fst.
chooseFiber / sigmaSection	classical section of the bundle.
sigmaSection_rightInverse_sigma_fst	RightInverse law.
canonicalSigmaFiberWitness	uniform witness in each SigmaFiber.

Table 3: T3 (dependent sum): architecture-guided bundle (SigmaFiberBenchmark.lean).

7 Comparative synthesis across tranches

T1 isolates *collapse by an equivalence relation*; **T2** isolates *multi-route* without quotienting; **T3** is the dependent-type generalization of T2. In all three cases the **STRONG** rubric applies for the same reason: the architecture-guided bundle is strictly richer than the flat baseline as a typed interface.

Tr.	Bare B	Projection π	What varies in the fiber	Section flavor
T1	quotient	`forgetToQuotient` / `Quotient.mk`	equivalence classes	`Quotient.out` (canonical)
T2	X	`forgetFirst`	second coordinate $y : Y$	`productSection y0` (parametric)
T3	B	`Sigma.fst`	dependent $E\,b$	`sigmaSection` (`noncomputable`)

Table 4: Side-by-side view: three instantiations of the same architectural schema.

8 Unified validation verdict and the meaning of STRONG

Under the project's validation rubric [3], a benchmark is rated **STRONG** when it exports the architecture cleanly to external mathematics and yields a theorem bundle structurally richer than the obvious baseline. Each tranche here satisfies that criterion on its own; *collectively*, they strengthen the methodological claim: the pattern is not tied to quotients alone.

STRONG encodes methodological transfer and corollary structure, not difficulty of raw mathematical content. No tranche proves a new "summit" theorem for the IC landscape; together they support external validity of the *packaging*.

9 Place in the larger program

- **IC flagship (internal):** the positive-closure architecture and main certification / reflective splits—load-bearing inside the library.
- **External validation (this paper):** machine-checked demonstrations that the same pattern applies *outside* the flagship modules, on quotients, products, and dependent sums.
- **Strata:** abstract synthesis and bridge laws [2].
- **APS:** local-to-global backbone elsewhere in the stack. **These tranches do not test APS directly;** they test reusable positive-closure *packaging* on external mathematics.
- **NEMS:** outer burden and interface story for the paper suite.

10 Limitations and future work

Three finite families do not exhaust all closure or fibration patterns. Natural extensions include: settings where sections are non-unique or only locally defined; competing enriched layers over one collapse map; and domain-specific benchmarks outside pure type theory. Such work would extend the *evidence base* without changing the logical status of the positive-closure architecture inside IC.

A Theorem indices (Lean names by module)

`QuotientFiberBenchmark.lean`

Lean name	One-line reading
`BareQuotient`	quotient type as bare certificate layer after collapse.
`EnrichedCarrier`	carrier before quotienting (enriched layer).
`forgetToQuotient`	projection `Quotient.mk` from carrier to quotient.
`QuotientFiber`	fiber over a bare quotient point (subtype of the carrier).
`quotient_fiber_nonempty`	each bare point has a nonempty fiber (from `exists_rep`).
`forgetToQuotient_surjective`	`forgetToQuotient` is surjective onto the quotient.
`forgetToQuotient_hasRightInverse`	projection has a global right inverse (section exists).
`quotientOut_rightInverse_forgetToQuotient`	`Quotient.out` is a right inverse of `forgetToQuotient`.
`canonicalQuotientFiberWitness`	canonical distinguished point in each `QuotientFiber`.
`quotient_fiber_nonempty'`	nonemptiness via `canonicalQuotientFiberWitness`.

`ProductProjectionFiberBenchmark.lean`

Lean name	One-line reading
`forgetFirst`	first projection $X \times Y \to X$.
`ProductFiberAt X Y x`	fiber of `forgetFirst` over `x`.
`productSection`	section $x \mapsto (x, y_0)$ at fixed `y0`.
`product_fiber_nonempty`	nonempty `ProductFiberAt` given `y0`.
`forgetFirst_surjective`	`forgetFirst` surjective given `y0`.
`forgetFirst_hasRightInverse`	`HasRightInverse` for `forgetFirst`.
`productSection_rightInverse_forgetFirst`	`RightInverse` law.
`canonicalProductFiberWitness`	canonical witness in each fiber.
`product_fiber_nonempty'`	nonemptiness via canonical witness.

`SigmaFiberBenchmark.lean`

Lean name	One-line reading
`SigmaFiber`	fiber of `Sigma.fst` over `b`.
`sigma_fiber_nonempty`	nonempty fibers from global nonemptiness.
`sigma_fst_surjective`	`Sigma.fst` surjective.
`sigma_fst_hasRightInverse`	`HasRightInverse`.
`chooseFiber`	`Classical.choice` in each fiber.
`sigmaSection`	global section of the bundle.
`sigmaSection_rightInverse_sigma_fst`	`RightInverse` law.
`canonicalSigmaFiberWitness`	canonical witness in each `SigmaFiber`.

References

[1] N. Spivack. *Canonical Certification Does Not Exhaust Reflective Structure: Reflective Split, Strict Refinement, and Fiber Structure in Infinity Compression*. Manuscript, 2026. Companion LaTeX source: `papers/Canonical_Certification_and_Enriched_Reflective_Realization/`.

[2] N. Spivack. *Strata: Abstract Synthesis and Bridge Laws (Infinity Compression)*. Companion manuscript in preparation, 2026.

[3] Infinity Compression contributors. *EPIC_015_WV1 — External Proof Architecture Validation* (specification and validation verdict notes). `infinity-compression` repository, `specs/` tree, 2026.

[4] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction — CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[5] The Mathlib community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020. DOI: 10.1145/3372885.3373824. Mathlib 4 repository: https://github.com/leanprover-community/mathlib4.

[6] S. Mac Lane. *Categories for the Working Mathematician*, chapter I (categories of algebraic structures; universal constructions motivating quotients). Springer, 2nd edition, 1998.